AI WORKFLOW DOCUMENT

How WorkRadar was built with AI

The tools, the prompts that mattered, what AI produced, what changed after review, and the trade-offs behind each decision.

Aditya Singh
Airtribe · AI Product Management
v1.1 · 21 October 2026

01 Overview

AI shows up in WorkRadar in two layers, and the project treats them differently.

AI inside the product. Google Gemini writes the manager's daily brief from the ranked snapshot. It runs through one serverless route on Vercel, so the API key never reaches the browser. It does *not* decide the ranking. A deterministic, explainable score does that, so every item can show why it is on the radar. Gemini answers under a JSON schema and its output is validated. If it fails, the app shows an on-device summary and says why.

AI used to build the product. Claude (Anthropic) was the build partner from brief to delivery. It framed the problem, drafted the PRD, set the visual direction, wrote the React app and the AI route, built the automated checks, generated the Figma file from the running app, and set up the Vercel deployment in my browser. I set the constraints and made the product calls: the Airtribe context, Gemini as the model provider, Vercel for hosting, and the scope. Every AI output went through a review pass before it shipped.

At a glance

7
AI and automation tools in the workflow

~2 s
live brief with Flash-Lite, against 35+ s with Flash

1
production prompt, schema-constrained and validated

3
engines in the fallback chain: Flash-Lite, Flash, on-device

15
hi-fi screens generated from the live app

13
changes made after reviewing AI output (section 06)

02 AI tools used

TOOL	ROLE	WHAT IT DID	WHY THIS TOOL
Claude (Anthropic), agentic coding environment	Build partner	Problem framing, personas, PRD and this document; design direction; React + TypeScript app; scoring model; the serverless AI route; automated checks; README; DOM-to-Figma pipeline	Long-context reasoning across product, design and code. It can run, test and screenshot its own work.
Google Gemini · 3.5 Flash-Lite, 3.8 Flash fallback	In-product LLM	Writes the daily brief from a ranked snapshot: headline, top three for today, what happened, needs attention, and who's blocked with a way to unblock them.	Native JSON-schema output (`responseSchema`), a free tier for the pilot, about 2 s per brief
Vercel · Edge Functions	Hosting and AI gateway	Serves the app. `/api/digest` calls Gemini with the key kept server-side, streams while the model thinks, and falls back between models.	Free, deploys on every push to GitHub, keeps the secret off the browser
Claude in Chrome	Deployment	Set up and deployed the Vercel project in my browser, then read the function logs that showed the slow model	Clicked through the setup for me; I only signed in and pasted the key
Figma MCP (Plugin API)	Design output	Rebuilt captured app states as editable Figma layers. A bundled plugin covers the remaining screens.	Design file and shipped product never drift
Playwright (headless Chromium)	Capture and QA	Renders 15 app states for the DOM extractor and screenshots, visual review at desktop and mobile widths, HTML-to-PDF for these documents	Repeatable visual checks without manual clicking
GitHub Actions	Verification	Typecheck, production build and 11 checks on every push: ranking, brief validation, search and the AI route	Catches AI-introduced regressions automatically

03 How the work flowed

1 · FRAME
Problem and users
Turned the brief into four manager questions, two personas and six pain points.
AI: drafts · **Me:** Airtribe context, scope

2 · DECIDE
Product calls
Explainable ranking, AI for language only, a fallback chain, mock data for the demo.
AI: options and trade-offs · **Me:** chose

3 · DESIGN
Visual direction
"Air-traffic control": radar scope hero, flight-strip rows, Barlow type, light and dark tokens.
AI: direction and CSS · **Me:** approved

4 · BUILD
App and AI route
React app, scoring, search parser, the Gemini route, 47 mock items relative to "now".
AI: code · **Me:** keys, accounts, repo

5 · VERIFY
Review loops
Screenshot review against the PRD, automated checks, a latency test on the live site. 13 changes.
AI: self-review · **Me:** acceptance

6 · SHIP
Deliverables
Live on Vercel, Figma from the app, PRD, this document, README, demo script.
AI: drafts, deploy · **Me:** final edit, demo

04 Where AI runs inside the product

SOURCES (DEMO: MOCK DATA)	IN THE APP	VERCEL · /API/DIGEST	MANAGER
Slack Threads, DMs, mentions. No priority field.	**One work-item shape** Owner, due date, links, blockers, signals, the same for all four tools	**Gemini 3.5 Flash-Lite · AI** Default. Prompt + JSON schema, about 2 s per brief	**WorkRadar app** Today, Priority feed, Item detail, Search, Team. Every score shows “why”.
Jira Tickets with P-levels, sprints, links	**Score and rank** 6 weighted signals → 0–100, P0 pinned ≥ 97, rings: now ≥ 75, today ≥ 50	**Gemini 3.8 Flash · AI** Fallback on any failure or timeout. Separate free-tier quota, slower	**Validation** Drops unknown item IDs, resolves people, caps lists, rejects briefs without a headline
GitHub PRs, reviews, checks	**Snapshot** Top 24 open items, blocked people and the last 24 hours, capped at 80 KB	**Streaming response** Starts at once and sends the JSON when ready, so a slow model never hits the platform’s time limit	**On-device summary** If Gemini fails: the same data summarised in the browser, the reason in plain words, and Try again
Gmail Escalations, stakeholder asks			

Green boxes use an LLM. Grey boxes are deterministic code. The split is deliberate: AI handles language, code handles ranking and anything that must be auditable.

05 Significant prompts

A · Daily brief

Where: `api/_prompt.ts`, sent by the `/api/digest` route

Model: Gemini 3.5 Flash-Lite (3.8 Flash fallback), temperature 0.3, `responseMimeType: application/json` + `responseSchema`

Design choices: facts-only rule, exact IDs so output can be validated, sentence-length caps so it fits the UI, and a fixed shape for each of the five sections.

```
You are WorkRadar, a chief-of-staff assistant for an engineering manager.
You receive a JSON snapshot of their team's work from Slack, Jira, GitHub and email.
Items are already ranked by a transparent priority score (0-100) and grouped into rings:
"now" (needs attention now), "today" and "later".
Write the manager's brief as JSON that matches the response schema.

Rules
- Use only facts in the snapshot. Never invent people, numbers, deadlines or items.
- Put the exact item "id" from the snapshot in every itemId field. Use team member ids for
personId.
- Plain, specific language. No hype, no emojis, no exclamation marks.
- Each "why", "action" and "suggestion" is one sentence under 25 words.

Fields
- headline: 1-2 sentences. How many items need attention, how many people are blocked
  (and how many on the manager), and the one thing to start with and why.
- topWork: exactly 3 items the manager should personally act on today ... "why" is the concrete
next step.
- happened: up to 6 notable events from happenedToday as short past-tense sentences.
- attention: every item in ring "now" (max 6), with why it matters now and the action to take.
- blocked: one entry per blocked person with personId, itemId, blockedOn, since and suggestion.
```

B · The project brief given to Claude (condensed)

It set the deliverables and the quality bar. Its last line made Claude plan first, which surfaced the questions that mattered: author, company, deadline and Figma access.

```
Build WorkRadar, an AI-powered web app that helps
managers see top work for the day, what happened today,
what needs immediate attention and who is blocked,
without checking Slack, Jira, GitHub or email.

Deliverables: PRD (PDF, max 2 pages) · hi-fi UI designs
(Dashboard, Priority Feed, Item Details, Search,
Empty/Error state) · working MVP (dashboard, AI
summary, priority feed, search, detail page, Assign /
Mark Done / Snooze; mock data is fine) · AI workflow
doc · source code · demo video.

I want a full-fledged, functioning, visually aesthetic
project, so use your best judgement for the design
tool. Before starting, prepare a list of tasks or
things you need from me.
```

C · Framing and constraints (my reply, lightly edited)

One reply reshaped the product. The Airtribe context gave the mock data its stakes: live classes, the GenAI Launchpad and mentor sessions.

```
1. Aditya Singh. Create this project for an Airtribe
manager itself, for best impact, as this assignment is
part of my AI product management course. Their website:
airtribe.live
2. The assignment is due on 16 Oct 2026.
3. I have connected GitHub with Claude, with an empty
repo called workradar.
4. Once the project is complete, ask me again (about
the demo video).
5. Yes, you can create a new file in my Figma drafts.
```

How review worked in the build loop: each screen was screenshotted and reviewed from a sceptical manager’s point of view at 9 am, asking what is wrong, confusing or untrustworthy. Every number in the brief was checked against the snapshot.

Prompt A is quoted from the repository (`api/_prompt.ts`); “…” marks shortening.

06 What AI generated, and what changed after review

AREA	GENERATED BY AI	CHANGED AFTER REVIEW
Ranking model	Six-signal weighted score, rings, "why" details per factor	→ The P0 iPhone incident ranked below a PR, so P0 is now pinned to ≥ 97. → Slack and email items read "No priority set". They now read "Inferred P1".
Brief copy	Prompt, schema, on-device brief	→ Absolute clock times contradicted relative times ("in 5h"), so they were removed from AI copy. → "1 days" became "about a day", and lowercase sentence starts were capitalised.
Validation	JSON schema output	→ Added `sanitizeBrief` : drops unknown item IDs, resolves names to people, caps list lengths, rejects briefs without a headline.
Model choice	Gemini 3.8 Flash as the default	→ On the live site it took over 35 s and hit the platform's response limit. Timed side by side, Flash-Lite answered in about 2 s, so it became the default and Flash the fallback.
AI route	Server-side Gemini call	→ The route now streams, so slow answers aren't cut off, and it falls back on timeouts, not just rate limits. → Added a same-origin check, an 80 KB payload cap and a 40-item cap. The key lives only in Vercel's environment variables.
AI failures	Error passed straight to the UI	→ Raw text like "Gemini 503: …" reached the brief. Failures now show one plain sentence, the on-device summary and Try again.
Priority feed	Flight-strip rows, views, filters, keyboard triage	→ The default view was too sparse, so it is now "All open" with ring headers. → Item refs were truncated, so columns were widened and DM refs shortened. Keyboard hints moved to their own line.
Empty states	First run, no results, nothing needs you	→ On first run, source icons stretched and the sidebar claimed sources had synced "2m ago". Fixed to "Not connected".
Scope	A workflow-automation layer (n8n)	→ It was built because the brief mentioned n8n, which turned out to be a mistake in the brief. It was removed, leaving one moving part: the serverless AI route.
Figma pipeline	DOM extractor and Figma renderer	→ Fixed text that truncated when it fit, missing underlines and percentage corner radii. Starter-plan limits led to a bundled plugin.

Written by hand, not generated: the choice of problem and users, the Airtribe framing, all product scope decisions in the PRD, the hosting choice, API keys and accounts, and final acceptance of each deliverable.

Prompting patterns that worked

- **Ask for a plan and questions first.** "Prepare a list of what you need from me" turned a vague brief into concrete decisions before any code was written.
- **Give the model the business context.** The prompt names the manager, the rings and what counts as needing attention, so the brief prioritises the way the team does.
- **Constrain output to the UI.** Schema fields map one-to-one to UI slots, with word limits sized to the layout.
- **Make hallucinations detectable.** Requiring exact item IDs means any invented item can be dropped automatically.
- **Pick the model by measuring, not by name.** The bigger model looked like the obvious choice. Timing both on the live site flipped the default.
- **Review against screenshots, not code.** Most quality issues (ranking surprises, copy, truncation) were only visible on the rendered screen.

07 Key trade-offs

DECISION	CHOSEN	INSTEAD OF	WHY, AND WHAT IT COSTS
Who ranks	Deterministic weighted score; AI for language	LLM ranks everything	Explainable, stable between refreshes, testable and free. It can't pick up subtle context, so every factor is shown and the weights are adjustable.
Output format	JSON schema + validation	Free-text summary	Renders into specific UI slots, links to real items and can be checked for invented IDs. Costs some expressiveness, mitigated by tight per-field instructions.
Architecture	One serverless route on Vercel	A workflow engine or a separate backend	One moving part to deploy and secure, free, and redeployed on every push. Scheduled jobs such as a morning digest would need a scheduler later.
Model	Gemini 3.5 Flash-Lite, with 3.8 Flash as fallback	Flash or a Pro model as the default	Measured about 2 s against 35+ s. The brief summarises an already-ranked list, so the lighter model is enough. Nuance on long threads is weaker.
Delivery	Stream the response at once	Wait, then reply	Slow models can't hit the edge time limit. The cost: errors travel inside a 200 response, so the client reads the body, not the status.
Availability	Flash-Lite → Flash → on-device	Single AI path	The brief is never empty and failures are explained. The rare failure path waits for a second model.
Data sent to AI	Top 24 open items + blocked people + last 24h	Every raw thread	Fewer tokens, less private content leaving the app, more focused output. Low-ranked items can't appear in the brief, which is acceptable by design.
Data	Mock workspace with timestamps relative to "now"	Live OAuth connectors	Allowed by the brief. The demo always reads as "today" and is safe to share. Real-world noise and edge cases are untested until the pilot.
Design file	Generated from the running app (DOM → Figma)	Mockups drawn first	Design and code never drift, and 15 states came cheaply. Layers are absolutely positioned rather than auto-layout components.
UI stack	Hand-written CSS tokens, no UI library	Component library	A distinctive look, a small bundle (85 KB gzip JS) and full control of light and dark. More code to maintain; AI made it cheap to write.

Guardrails and how the AI is checked

- **Facts-only prompting** with exact IDs, then **validation** that drops anything the snapshot can't support.
- **Every score is explained** by factor in the UI, so the manager can disagree and re-weight.
- **An engine label on every brief** (Gemini or on-device), so users know what they're reading.
- **Automated checks in CI** (`npm test`): the expected top three (iPhone incident 97, PR #1284 93, LP-512 86), brief validation, search parsing, and the route's fallback, origin and size rules.
- **Graceful failure:** the outage banner hides a down source and explains the effect on ranking. AI failures show the reason, the on-device summary and Try again.
- **Secrets** live only in Vercel's environment variables. The browser never sees the key.

What I'd do next with AI

- **An eval set:** 50 golden snapshots with expected top-3 lists, plus an LLM-as-judge faithfulness check on every brief, tracked weekly.
- **Live triage:** once real connectors exist, Gemini reads Slack and email threads and sets priority, owner and next step.
- **Learned weights:** tune the six weights per manager from opens, snoozes and reorders, keeping the explanation.
- **Reply drafts:** Gemini drafts responses for "waiting on you" asks, sent only after the manager approves.
- **Write-back** to Jira, GitHub and Slack, with an audit log and undo.
- **A privacy layer:** redact personal data before anything is sent to Gemini, and keep message bodies for at most 7 days.

Lesson The most useful AI work wasn't generating code. It was generating options and then critiquing the result from the user's point of view, on real screenshots and on the live site. Measuring beat assuming: the "better" model was the wrong default.

Links Live app: work-radar-ten.vercel.app · Source: github.com/adityasingh535402-del/WorkRadar · Designs: figma.com/design/Q8RHXzlj6yOqBR4HDq5hn5